2.1 Pablo DEX 팰릿 분석

개요

Pablo DEX는 Composable Finance의 핵심 탈중앙화 거래소(DEX) 팰릿으로, Automated Market Maker(AMM) 프로토콜을 구현합니다. 이 팰릿은 Balancer V1 스타일의 Constant Product 풀을 기반으로 하며, 유동성 제공, 토큰 스왑, Time-Weighted Average Price(TWAP) 기능을 제공합니다.

주요 구성 요소

2.1.1 팰릿 구조

Pablo 팰릿은 다음과 같은 주요 모듈로 구성됩니다:

- **lib.rs**: 메인 팰릿 로직 및 extrinsic 함수들
- **dual_asset_constant_product.rs**: Constant Product AMM 구현
- **twap.rs**: Time-Weighted Average Price 계산 로직
- **types.rs**: 데이터 타입 정의

2.1.2 핵심 데이터 구조

PoolConfiguration

```
#[derive(RuntimeDebug, Encode, Decode, MaxEncodedLen, CloneNoBound, PartialEqNoB
pub  enum PoolConfiguration<AccountId: Clone + PartialEq + Debug, AssetId: Clone
DualAssetConstantProduct(BasicPoolInfo<AccountId, AssetId, ConstU32<2>>),
```

```
}
```

PoolInitConfiguration

```
#[derive(RuntimeDebug, Encode, Decode, MaxEncodedLen, CloneNoBound, PartialEq, E
pub  enum PoolInitConfiguration<AccountId: Clone, AssetId: Clone> {
DualAssetConstantProduct {
owner: AccountId,
assets_weights: Vec<(AssetId, Permill)>,
fee: Permill,
},
}
```

2.1.3 스토리지 구조

Pablo 팰릿은 다음과 같은 스토리지 아이템들을 사용합니다:

PoolCount

```
#[pallet::storage]
#[pallet::getter(fn pool_count)]
pub  type PoolCount<T: Config> = StorageValue<_, T::PoolId, ValueQuery, PoolCoun
```

- **목적**: 생성된 풀의 총 개수를 추적

- **타입**: `StorageValue`
- **기본값**: `Zero::zero()`

Pools

```
#[pallet::storage]
#[pallet::getter(fn pools)]
pub type Pools<T: Config> = StorageMap<_, Blake2_128Concat, T::PoolId, PoolConf
```

- **목적**: 풀 ID를 키로 하여 풀 설정 정보를 저장
- **타입**: `StorageMap`
- **키**: `T::PoolId`
- **값**: `PoolConfigurationOf<T>`

TWAPState

```
#[pallet::storage]
#[pallet::getter(fn twap)]
pub type TWAPState<T: Config> = StorageMap<_, Blake2_128Concat, T::PoolId, TWAP
```

- **목적**: 각 풀의 TWAP 상태 정보를 저장
- **타입**: `StorageMap`
- **키**: `T::PoolId`
- **값**: `TimeWeightedAveragePrice` 구조체

PriceCumulativeState

```
#[pallet::storage]
#[pallet::getter(fn price_cumulative)]
pub type PriceCumulativeState<T: Config> = StorageMap<_, Blake2_128Concat, T::P
```

- **목적**: 가격 누적 상태를 저장하여 TWAP 계산에 사용
- **타입**: `StorageMap`
- **키**: `T::PoolId`
- **값**: `PriceCumulative` 구조체

2.1.4 주요 Extrinsic 함수들

create

```
#[pallet::call_index(0)]
#[pallet::weight(T::WeightInfo::create())]
pub fn create(origin: OriginFor<T>, pool: PoolInitConfigurationOf<T>) -> Dispa
```

- **목적**: 새로운 유동성 풀을 생성
- **매개변수**:
- `origin` : 풀 생성 권한을 가진 계정
- `pool` : 풀 초기화 설정
- **이벤트**: `PoolCreated`

swap

```
#[pallet::call_index(2)]
#[pallet::weight(T::WeightInfo::swap())]
pub fn swap(
origin: OriginFor<T>,
pool_id: T::PoolId,
in_asset: AssetAmount<T::AssetId, T::Balance>,
min_receive: AssetAmount<T::AssetId, T::Balance>,
keep_alive: bool,
) -> DispatchResult
```

- **목적**: 토큰 스왑 실행
- **매개변수**:
- `pool_id` : 스왑을 실행할 풀 ID
- `in_asset` : 입력 자산과 수량
- `min_receive` : 최소 수령 자산과 수량
- `keep_alive` : 계정 생존 여부
- **이벤트**: `Swapped`

buy

```
#[pallet::call_index(1)]
#[pallet::weight(T::WeightInfo::buy())]
```

```
pub fn buy(
origin: OriginFor<T>,
pool_id: T::PoolId,
in_asset_id: T::AssetId,
out_asset: AssetAmount<T::AssetId, T::Balance>,
keep_alive: bool,
) -> DispatchResult
```

- **목적**: 특정 수량의 토큰을 구매
- **매개변수**:
- `in_asset_id` : 지불할 자산 ID
- `out_asset` : 구매할 자산과 수량
- **이벤트**: `Swapped`

add_liquidity

```
#[pallet::call_index(3)]
#[pallet::weight(T::WeightInfo::add_liquidity())]
pub fn add_liquidity(
origin: OriginFor<T>,
pool_id: T::PoolId,
assets: BTreeMap<T::AssetId, T::Balance>,
min_mint_amount: T::Balance,
keep_alive: bool,
```

```
) -> DispatchResult
```

- **목적**: 풀에 유동성 추가
- **매개변수**:
- `assets` : 추가할 자산들과 수량
- `min_mint_amount` : 최소 LP 토큰 발행량
- **이벤트**: `LiquidityAdded`

remove_liquidity

```
#[pallet::call_index(4)]
#[pallet::weight(T::WeightInfo::remove_liquidity())]
pub fn remove_liquidity(
origin: OriginFor<T>,
pool_id: T::PoolId,
lp_amount: T::Balance,
min_receive: BTreeMap<T::AssetId, T::Balance>,
) -> DispatchResult
```

- **목적**: 풀에서 유동성 제거
- **매개변수**:
- `lp_amount` : 소각할 LP 토큰 수량
- `min_receive` : 최소 수령 자산 수량들
- **이벤트**: `LiquidityRemoved`

2.1.5 AMM 구현 (DualAssetConstantProduct)

핵심 수학 공식

Pablo DEX는 Constant Product 공식을 사용합니다:

```
x * y = k (상수)
```

여기서:

- `x`, `y` : 풀 내 두 자산의 수량
- `k` : 상수 (불변량)

주요 함수들

do_create_pool

```
pub(crate) fn  do_create_pool(
who: &T::AccountId,
fee_config: FeeConfig,
assets_weights: BoundedBTreeMap<T::AssetId, Permill, ConstU32<2>>,
lp_token_id: Option<AssetIdOf<T>>,
) -> Result<(T::PoolId, AssetIdOf<T>), DispatchError>
```

- **목적**: 새로운 Constant Product 풀 생성
- **검증사항**:
- 정확히 2개의 자산
- 가중치가 0이 아님

- 가중치 합이 1
- 수수료율이 100% 미만

get_exchange_value

```
pub(crate) fn  get_exchange_value(
pool: &BasicPoolInfo<T::AccountId, T::AssetId, ConstU32<2>>,
pool_account: &T::AccountId,
in_asset: AssetAmount<T::AssetId, T::Balance>,
out_asset_id: T::AssetId,
apply_fees: bool,
) -> Result<(AssetAmount<T::AssetId, T::Balance>, AssetAmount<T::AssetId, T::Bal
```

- **목적**: 스왑 시 교환 비율 계산
- **사용 공식**: `compute_out_given_in` (Balancer V1 공식)

add_liquidity

```
pub(crate) fn  add_liquidity(
who: &T::AccountId,
pool: BasicPoolInfo<T::AccountId, T::AssetId, ConstU32<2>>,
pool_account: T::AccountId,
assets: BiBoundedVec<AssetAmount<T::AssetId, T::Balance>, 1, 2>,
min_mint_amount: T::Balance,
keep_alive: bool,
```

```
) -> Result<(T::Balance, BTreeMap<T::AssetId, T::Balance>), DispatchError>
```

- **목적**: 유동성 추가 처리
- **로직**:

1. 단일 자산 예치 vs 다중 자산 예치 구분
2. 첫 예치 vs 추가 예치 구분
3. LP 토큰 발행량 계산
4. 자산 전송 및 LP 토큰 발행

2.1.6 TWAP (Time-Weighted Average Price) 시스템

TWAP 데이터 구조

TimeWeightedAveragePrice

```rust
#[derive(Encode, Decode, TypeInfo, Clone, Default, PartialEq, Eq, RuntimeDebug)]
pub struct TimeWeightedAveragePrice<Timestamp, Balance> {
pub timestamp: Timestamp,
pub base_price_cumulative: Balance,
pub quote_price_cumulative: Balance,
pub base_twap: Rate,
pub quote_twap: Rate,
}
```

PriceCumulative

```
#[derive(Encode, Decode, TypeInfo, Clone, Default, PartialEq, Eq, RuntimeDebug)]
pub struct PriceCumulative<Timestamp, Balance> {
pub timestamp: Timestamp,
pub base_price_cumulative: Balance,
pub quote_price_cumulative: Balance,
}
```

TWAP 계산 로직

enable_twap

```
#[pallet::call_index(5)]
#[pallet::weight(100_000)]
#[transactional]
pub fn enable_twap(origin: OriginFor<T>, pool_id: T::PoolId) -> DispatchResult
```

- **목적**: 특정 풀에 대해 TWAP 기능 활성화
- **초기화**: 현재 가격을 기준으로 누적 가격 상태 초기화

update_price_cumulative_state

```
pub(crate) fn update_price_cumulative_state<T: Config>(
pool_id: T::PoolId,
prev_price_cumulative: &mut Option<PriceCumulativeStateOf<T>>,
```

```
) -> Result<(T::Balance, T::Balance), DispatchError>
```

- **목적**: 가격 누적 상태 업데이트
- **계산**: `price_cumulative += current_price * time_elapsed`

update_twap_state

```
pub(crate) fn update_twap_state<T: Config>(
base_price_cumulative: T::Balance,
quote_price_cumulative: T::Balance,
prev_twap_state: &mut Option<TWAPStateOf<T>>,
) -> Result<(), DispatchError>
```

- **목적**: TWAP 값 계산 및 업데이트
- **조건**: `elapsed_time >= TWAPInterval`
- **계산**: `twap = (current_cumulative - previous_cumulative) / elapsed_time`

블록 훅에서의 TWAP 업데이트

```
#[pallet::hooks]
impl<T: Config> Hooks<T::BlockNumber> for Pallet<T> {
fn on_initialize(_block_number: T::BlockNumber) -> Weight {
let twap_enabled_pools: Vec<T::PoolId> = PriceCumulativeState::<T>::iter_keys()
for pool_id in twap_enabled_pools {
// TWAP 상태 업데이트 로직
}
```

```
}

}
```

2.1.7 이벤트 시스템

PoolCreated

```
PoolCreated {

pool_id: T::PoolId,

owner: T::AccountId,

asset_weights: BTreeMap<T::AssetId, Permill>,

lp_token_id: T::AssetId,

}
```

Swapped

```
Swapped {

pool_id: T::PoolId,

who: T::AccountId,

base_asset: T::AssetId,

quote_asset: T::AssetId,

base_amount: T::Balance,

quote_amount: T::Balance,
```

```
fee: Fee<T::AssetId, T::Balance>,

}
```

LiquidityAdded

```
LiquidityAdded {

who: T::AccountId,

pool_id: T::PoolId,

asset_amounts: BTreeMap<T::AssetId, T::Balance>,

minted_lp: T::Balance,

}
```

TwapUpdated

```
TwapUpdated {

pool_id: T::PoolId,

timestamp: MomentOf<T>,

twaps: BTreeMap<T::AssetId, Rate>,

}
```

2.1.8 수수료 시스템

Pablo DEX는 다음과 같은 수수료 구조를 사용합니다:

FeeConfig

```
pub struct FeeConfig {
pub fee_rate: Permill,
pub owner_fee_rate: Permill,
pub protocol_fee_rate: Permill,
}
```

수수료 계산 및 분배

```
fn disburse_fees(
who: &T::AccountId,
_: &T::PoolId,
owner: &T::AccountId,
fees: &Fee<T::AssetId, T::Balance>,
) -> Result<(), DispatchError>
```

- **풀 소유자 수수료**: 풀 생성자에게 지급
- **프로토콜 수수료**: 프로토콜 금고에 적립

2.1.9 보안 및 검증

주요 검증 사항

1. **풀 존재 확인**: `get_pool()` 함수로 풀 존재성 검증

2. **자산 유효성**: 풀에 포함된 자산인지 확인

3. **유동성 충분성**: 스왑 시 충분한 유동성 확인

4. **최소 수령량**: 슬리피지 보호를 위한 최소 수령량 검증

5. **동일 자산 스왑 방지**: `CannotSwapSameAsset` 에러로 방지

트랜잭션 안전성

모든 주요 함수는 `#[transactional]` 매크로를 사용하여 원자성을 보장합니다:

```
#[transactional]
fn do_swap(...) -> Result<SwapResult<...>, DispatchError>
```

2.1.10 성능 최적화

가중치 기반 계산

- Balancer V1 스타일의 가중치 기반 AMM으로 다양한 비율의 풀 지원
- 50:50 풀뿐만 아니라 80:20, 60:40 등 다양한 비율 지원

배치 처리

- `on_initialize` 훅에서 모든 TWAP 활성화 풀을 배치로 업데이트
- 가스 효율성을 위한 최적화된 스토리지 접근

시뮬레이션 기능

```
fn simulate_add_liquidity(...) -> Result<Self::Balance, DispatchError>
```

```
fn  simulate_remove_liquidity(...) -> Result<BTreeMap<Self::AssetId, Self::Balan
```

- 실제 실행 전 결과 미리보기 가능
- 프론트엔드에서 사용자 경험 향상

이 Pablo DEX 팰릿은 Composable Finance 생태계의 핵심 DeFi 인프라로서, 안전하고 효율적인 토큰 교환 및 유동성 제공 서비스를 제공합니다.